RABBITMQ + АЛЬТЕРНАТИВИ: NATS І SQS

Замовлення №42: exchange → binding → ack → prefetch → DLQ

Заняття №19 | Node.js Pro

Лектор
Кирило Орлов

r_d

ЗАМОВЛЕННЯ №42: ПОДІЯ ЇДЕ ЧЕРЕЗ БРОКЕРА

Перша половина — механіка доставки	Друга половина — коли обробка не вдалась
крок 0 · топологія і дефолти з живої ноди	крок 5 · DLQ: x-death, delivery-limit, at-most-once
кроки 1-2 · autoAck губить 4 · ручний ack не губить	крок 6 · дубль є — ефект один: ідемпотентність
крок 3 · direct / topic / fanout на одному наборі ключів	кроки 7-8 · NATS JetStream і SQS поруч
крок 4 · prefetch: розподіл 20 / 0 і 10 / 10	ack — не «дійшло», а «повтору більше не треба»

НАСКРІЗНА НИТКА

Одна подія на весь вечір: «замовлення 42 оформлено». Питання одне: як вона доїде до п'яти споживачів так, щоб жоден не загубив її і жоден не спрацював двічі? Дев'ять кроків TypeScript на портах 21900–21920; усі виводи в деці — з реальних запусків code/19_Rabbitmq.

ПЛАН ЗАНЯТТЯ



- Черга — про роботу, лог — про факти
- Exchange, binding, routing key
- Чотири типи exchange і збіг ключа
- ack / nack / reject: хто відповідає
- Prefetch: дефолт 0 означає БЕЗ ЛІМІТУ
- Типи черг 4.x · DLQ як окремий контур
- Exactly-once чесно · NATS і SQS · ДЗ

СЬОГОДНІ ПОБАЧИТЕ НА ВЛАСНІ ОЧІ

01. 4 зникли

autoAck і SIGKILL: п'ять доставок, один ефект, чотири повідомлення без сліду

02. 2 викинуто

публікація підтверджена, а в чергах нічого: ключ без рядка в таблиці бінда

03. 20 / 0

дефолтний prefetch: перший воркер забрав усі двадцять, другий — нічого

04. DLQ 1 / робоча 0

reject із requeue=false: причину смерті читаємо із заголовка x-death

05. delivery-count = 1

дванадцять повернень через nack — лічильник стоїть, у DLQ нуль

06. дубль ≠ ефект

5 подій → 8 доставок → 5 ефектів: змінився лише споживач

ПРОБЛЕМА: П'ЯТЬ HTTP-ВИКЛИКІВ У РЯДОК

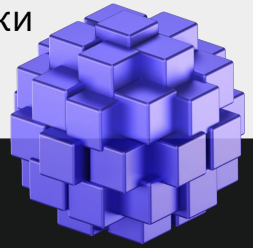
залишок, пакування, накладна, сповіщення, індекс — п'ять викликів

латентність = сумі всіх п'яти: найповільніший сусід — ваша сторінка

доступність = добутку: п'ять залежностей по 99.9% дають 99.5%

лежить пошук — не оформлюється замовлення; пік іде наскрізь

Теза вечора: брокер не доставляє — він зберігає, поки хтось не візьме відповідальність за повідомлення



```
КУДИ ДІВСЯ КОНВЕРТ?

продюсер   брокер
________   ______
publish  →
       ← ack
publish  →
       ← ack
 binding немає!
publish  →
       ← ack теж

_________________
повідомлення
викинуто мовчки,
а confirm прийшов
ПОЗИТИВНИЙ
```

ЧЕРГА — ПРО РОБОТУ, ЛОГ — ПРО ФАКТИ

```
ЧЕРГА (work)                ЛОГ (facts)
──────────                  ──────────
«зроби це» — адресоване     «це сталося» — ні до кого
взяв — листка немає         читання нічого не видаляє
один споживач на нього      скільки завгодно читачів
є природний DLQ             DLQ немає: факт не може
завдання може не вдатись    «не вдатися» — він стався
брокер тримає стан          споживач тримає офсет сам

// Де ламається черга: коли завдання стає ФАКТОМ, а
// список зацікавлених наперед НЕВІДОМИЙ — склад,
// сповіщення, аналітика, антифрод, пошук. Кожен новий
// зацікавлений = зміна топології або зміна продюсера.
// Саме тут починається наступна лекція — L#20.
//
// Анти-патерн: fan-out як «продюсер публікує N разів
// у N черг». Це та сама сильне зчеплення, від якої
// брокер мав врятувати, просто перефарбована.
```

Кошик із листками на стіні цеху: узяв — листка вже немає

1 · один споживач на повідомлення; двоє — це дві копії

2 · брокер пам'ятає стан кожного листка: узяли, повернули

3 · перечитати вчорашнє неможливо: листок зняли зі стіни

у черзі повідомлення адресоване: «зроби це», і воно зникає

урок: у черги є природний DLQ — завдання може не вдатися



ПОШТОВЕ ВІДДІЛЕННЯ: ЧОТИРИ СУТНОСТІ І ЇХНІ ВЛАСНИКИ

КОНЦЕПТ І ВЛАСНИК

- exchange — інфраструктура
- binding — інфраструктура
- routing key — ВІДПРАВНИК
- queue — інфраструктура
- default exchange — цукор

АНАЛОГІЯ (ПОШТА)

- сортувальний вузол
- рядок «індекс → скринька»
- індекс, написаний на конверті
- скринька
- sendToQueue('q') = publish('', 'q')

Де аналогія ламається: лист із невідомим індексом пошта хоч поверне відправнику, а брокер його викидає мовчки — і відповідає продюсеру позитивним confirm. Головне речення секції: індекс на конверті нічого не означає, поки в таблиці сортування немає рядка з цим індексом.



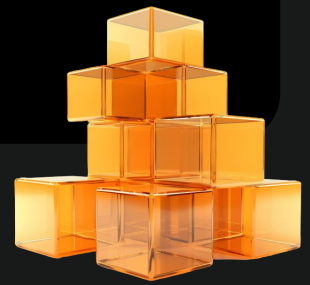
ТИХЕ ЗНИКНЕННЯ: CONFIRM ПОЗИТИВНИЙ, А ПОВІДОМЛЕННЯ НІДЕ

- routing key без рядка в таблиці бінда — повідомлення нікуди
- брокер відповідає basic.ack: публікація ПІДТВЕРДЖЕНА
- mandatory за замовчуванням false → повідомлення відкидають
- DLX не рятує: DLX живе НА ЧЕРЗІ, а до черги воно не дійшло
- ні помилки, ні логу, ні DLQ — найдорожчі граблі лекції
- три лікування: mandatory · alternate exchange · метрика

```
КРОК 3 · РЕАЛЬНИЙ ПРОГІН, ТАКТИ 3.5-3.7
$ npm run step3
3.5 брокер відповів basic.ack

опубліковано 4, розкладено 2
→ 2 ВИКИНУТО (немає binding)
basic.return отримано: 0

3.6 mandatory:true
basic.return отримано: 1
3.7 alternate exchange
l19.ae.catch: 1 — спіймано
──────────────────────────────────
повідомлення немає ніде: ні
помилки, ні логу, ні DLQ.
DLX не спрацює — він живе
на ЧЕРЗІ, а її тут не було
```



ДОСЛІВНО З ДОКУМЕНТАЦІЇ RABBITMQ



«When a published message cannot be routed to any queue,
 and the publisher set the mandatory message property
 to false (this is the default), the message is
 discarded or republished to an alternate exchange,
 if any.»
— rabbitmq.com/docs/publishers

«For unroutable messages, the broker will issue a
 confirm once the exchange verifies a message won't
 route to any queue (returns an empty list of queues).»
— rabbitmq.com/docs/confirms

// Брокер сказав правду. Він ПРИЙНЯВ відповідальність —
// і його відповідальність за немаршрутизоване
// повідомлення полягає рівно в тому, щоб його викинути.
//
// «Куди піде повідомлення, якщо я помилився в routing
// key?» — Нікуди. І мовчки: mandatory=false — це дефолт.

```
// src/00-setup.ts — ПОРЯДОК ОГОЛОШЕННЯ ТОПОЛОГІЇ:
// exchange → queue → binding → consumer → і лише
// ПОТІМ продюсер. Інакше — вікно тихого зникнення.
const conn = await amqp.connect(RABBIT_URL);  // :21900
const ch = await conn.createConfirmChannel();

await ch.assertExchange('l19.direct', 'direct', {
  durable: true,
  alternateExchange: 'l19.ae',   // лікування №2
});
await ch.assertQueue('l19.d.eu', {
  durable: true,
  arguments: { 'x-queue-type': 'quorum' },
});                    // classic з 4.0 не реплікується
await ch.bindQueue('l19.d.eu', 'l19.direct', 'order.eu');
// публікація: confirm-канал + mandatory
ch.publish('l19.direct', 'order.eu', payload, {
  persistent: true, mandatory: true, messageId: id });
ch.on('return', (m) => warn('basic.return', m.fields));
await ch.waitForConfirms();  // confirm ≠ «доставлено»
```



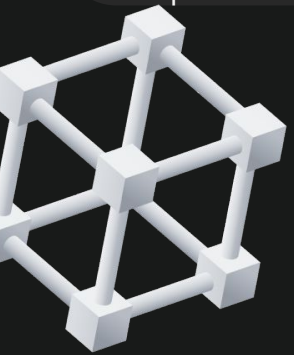
ТРИ СПОСОБИ ЗАКРИТИ ДІРУ — І ЩО КОЖЕН КОШТУЄ

mandatory: true — брокер поверне повідомлення через basic.return; ціна: треба написати обробник, без нього стане гірше, ніж було

alternate exchange — «або-інакше»-вихід у самого exchange; лишається інфраструктурою, продюсера не чіпаємо ⬅ дефолтний вибір

метрика unroutable — лічильник викинутих у management UI; не рятує, але робить видимим: потрібна В ПАРІ з попередніми

Порядок оголошення: exchange → queue → binding → consumer — і лише потім продюсер. Топологію оголошує той, хто споживає, або міграція, але не продюсер «на всяк випадок». Політики замість x-arguments: аргумент черги не змінити, не видаливши чергу



ТИП EXCHANGE — ЦЕ АЛГОРИТМ ЧИТАННЯ ТАБЛИЦІ БІНДА

Тип	Правило збігу	Коли брати
direct	точний збіг ключа з ключем binding	адресна робота: invoice.generate
fanout	копія в КОЖНУ чергу; ключ ігнорується	broadcast: «конфіг змінився»
topic	патерн: * — один сегмент, # — нуль або більше	90% реальних топологій
headers	збіг за мапою заголовків: x-match all | any	кілька осей маршрутизації — рідко
x-modulus-hash	плагін: розкидає по N черг за хешем	рукотворні партиції + SAC — у L#20 вбудовано



```
КРОК 3 · ОДИН ПРОДЮСЕР, ОДИН НАБІР КЛЮЧІВ — ТРИ КАРТИНИ

── 3.4                направлено  викинуто  копій
   direct  (2 binding)      2         2        2
   topic   (3 binding)      4         0        9
   fanout  (3 binding)      4         0       12
 √ продюсер у всіх трьох виконав ОДИН І ТОЙ САМИЙ код

── 3.2 · topic: збіг за патерном
   l19.t.eu      [order.eu.#     ] 3
   l19.t.created [order.*.created] 2
   l19.t.all     [order.#        ] 4
 ⚠️ order.eu.created.v2 НЕ потрапив у order.*.created:
   * — це рівно один сегмент, а тут їх два

// fanout: чотири повідомлення дали ДВАНАДЦЯТЬ копій.
// Це не «одне повідомлення трьом» — це три повідомлення
// з трьома незалежними ack. Оце і є ціна, яку в
// наступній лекції знімає лог.
```

ACK / NACK / REJECT: П'ЯТЬ ФАКТІВ, ЯКІ ВИРІШУЮТЬ

01. ack-режим — на basic.consume, на ВСІЙ підписці, не на повідомленні

02. ack — не «дійшло», а «я більше не вимагаю повтору»

03. ack(multiple: true) — граната: підтверджує все до тега включно

04. consumer_timeout стоково 30 хв; не встиг — requeue ВСЬОГО каналу

05. autoAck «щоб швидше» = at-most-once: 5 доставок, 1 ефект, 4 зникли

ЩО ІНКРЕМЕНТИТЬ ЛІЧИЛЬНИК НА 4.3

Виклик amqplib	Інкрементить delivery-count?	Впирається в delivery-limit?
reject(msg, true)	✅ так	✅ так — на N-й спробі йде в DLX
nack(msg, false, true)	✕ НЕ інкрементить	✕ ні — не обмежується взагалі
consumer_timeout (30 хв)	✕ НЕ інкрементить	✕ requeue ВСІХ unacked каналу
обрив каналу / SIGKILL	✕ ні: росте лише acquired-count	✕ ні — requeue всіх unacked
nack(msg, false, false)	— повідомлення одразу виходить	✅ в DLX, reason: rejected

PREFETCH: ДЕФОЛТ 0, І НУЛЬ ОЗНАЧАЄ БЕЗ ЛІМІТУ

basic.qos — це вікно НЕПІДТВЕРДЖЕНИХ доставок на споживача

з живої ноди: default_consumer_prefetch {ok,{false,0}} — БЕЗ ЛІМІТУ

ліміт НА КОЖНОГО споживача (global=false), а не на канал

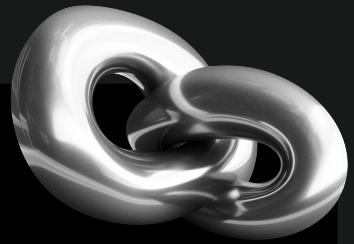
дослівно: «blindly dispatches every n-th message» — розкладає наперед

формула: prefetch ≈ час обробки / RTT до брокера, округлити вниз

перевір: prefetch × час обробки < consumer_timeout · стеля 2000

ТИПИ ЧЕРГ У 4.x І ЩО СТАЛОСЬ ІЗ ПОРЯДКОМ

ТИП / ФАКТ	ЩО ЦЕ ОЗНАЧАЄ
Classic (з 4.0)	не реплікується взагалі — вузька ніша
Quorum — дефолт лекції	Raft · poison handling · DLX at-least-once
Stream	лог з офсетами: читання НЕ видаляє
ha-mode / mirrored queues	прибрано з 4.0 — маркер статті до 2024
Порядок ДОСТАВКИ	ламають пріоритети і redelivery



```
КРОК 5 · DLQ — ОКРЕМИЙ КОНТУР, А НЕ СМІТНИК
// чотири причини, п'ятої немає: rejected · expired
// (message TTL) · maxlen · delivery_limit
// ⚠️ queue TTL НЕ dead-letter-ить: черга зникає з усім

── 5.3  робоча черга: 0 · DLQ: 1
   x-first-death-reason : rejected   ← перша смерть,
   x-first-death-queue  : l19.step5.work  не змінюється
   x-acquired-count     : 1
   x-delivery-count     : 1  ← за ЦИМ рахується ліміт
── 5.4  reject(requeue=true): доставок 4, у DLQ 1
   reason: delivery_limit · ліміт 3, а доставок 4
── 5.5 ⭐ nack(requeue=true): доставок 12+, у DLQ 0
   x-delivery-count = 1 — ліміт не спрацював ЖОДНОГО разу
── 5.7  брокер у логах:
   Falling back to dead-letter-strategy at-most-once …
   because … incompatible with effective overflow
   strategy drop-head. To enable … set overflow
   strategy to reject-publish.
// overflow: reject-publish — не «опція», а ОБОВ'ЯЗКОВА пара
```

```
ГОЛОВНА ФРАЗА ЛЕКЦІЇ ПРО DEAD-LETTERING
«dead-lettered messages are re-published without
 publisher confirms turned on internally… Messages are
 removed from the original queue immediately after
 publishing to the DLX target queue… messages can be
 lost if the target queue is not available.»
                        — rabbitmq.com/docs/dlx

at-most-once            може загубитись   ← autoAck
at-least-once           може прийти двічі ← ручний ack
exactly-once ДОСТАВКИ   НЕ ІСНУЄ НІ В КОГО
exactly-once ЕФЕКТУ     at-least-once + ідемпотентна
                        операція у споживача

// Ідемпотентність — властивість ОПЕРАЦІЇ: вона або є,
// або ні. UPDATE … SET status='packed' — є;
// UPDATE … SET qty = qty - 1 — немає.
// Ключ ідемпотентності — ІНСТРУМЕНТ: ним неідемпотентну
// операцію роблять безпечною для повтору, і його треба
// ПОБУДУВАТИ. Це різні речі, і плутають їх найчастіше.
```

ТРИ МІСЦЯ НЕВИЗНАЧЕНОСТІ — І ОДИН ПРИЙОМ НА ВСІ ТРИ

confirm

продюсер → брокер: confirm загубився — ретрай дає ДУБЛЬ у черзі. Confirms закривають втрату, не дубль

ack

брокер → споживач: ack загубився — робота виконується вдруге. Такт 4.6 показав це живцем

ідемпотентність

споживач → зовнішня система: записали в Postgres і впали до ack. Усі три закриває список оброблених id

ДВІ СУСІДНІ СТОРІНКИ ОДНОГО AWS DEVELOPER GUIDE
«Exactly-once processing — FIFO queues deliver each message once...» (sqs-queue-types) — і поруч: «at-least-once delivery model... no absolute guarantee that a message won't be delivered more than once» (sqs-visibility-timeout)

КАРТА РІШЕНЬ: ВІД ЗАДАЧІ ДО БРОКЕРА

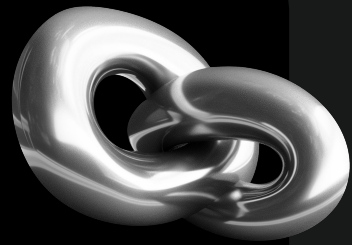
багата маршрутизація + DLQ → RabbitMQ 4.3
4 типи exchange, DLX із коробки
дедуплікації немає ВЗАГАЛІ

легко, багато інсталяцій → NATS JetStream
MaxAckPending 1000 СПІЛЬНИЙ на всіх
subject — рівно один потік (10065)

уже в AWS, нуль адміну → AWS SQS
немає ack: Delete або мовчання
порядок у межах MessageGroupId
розмір 1 МіБ, а не 256 КБ

перечитати минуле → це вже НЕ черга — L#20

20 msg/s, один споживач → Postgres SKIP LOCKED
чесніше за «брокер на виріст»



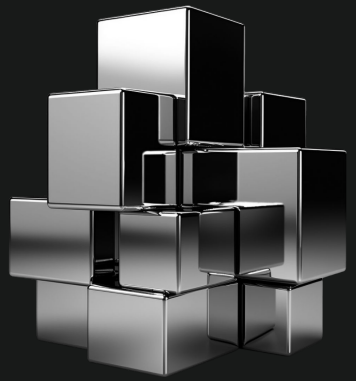
── ОДНА ІДЕЯ, ТРИ РІВНІ ──
prefetch (споживач) · MaxAckPending (споживач)
· SQS fair queues (тенант)

ЩО СВІДОМО ЛИШАЄМО ЗА ДУЖКАМИ

консенсус і Raft усередині quorum-черг: у нас ОДНА нода.
Адміністрування кластера, федерація, shovel — поза курсом.

Kafka → L#20 · ідемпотентність як ПАТЕРН (processed_messages, outbox, retry з джитером, DLQ як процес) → L#22 · Redis і BullMQ → L#23 · автентифікація підключень → L#24

⚠ стенд ≠ AWS: піднято ElasticMQ — реалізацію ПРОТОКОЛУ SQS; числові дефолти AWS цитуються. consumer_timeout занижений до 10 с проти стокових 30 хв — названо явно



ПІДСУМКИ + МІСТ ДО L#20

ЧОТИРИ РІШЕННЯ

binding, prefetch, ack, DLX — чотири рішення. Три з чотирьох мають дефолт, який у проді не годиться

ЩО КУПИЛИ

брокер дає буфер і маршрутизацію — і чесно каже, де закінчуються його гарантії

далі

L#20 — Kafka: сьогодні ack ВИДАЛЯВ повідомлення, наступного разу commit не видалятиме нічого

ДЗ #19

7 балів (PROJECT MEDIUM) · дедлайн 13.10 · дев'ять пунктів: топологія · publish із confirm · ручний ack · prefetch із числом · DLQ і x-death · ідемпотентний споживач · три демо-скрипти

ЧЕСНА МЕЖА

exactly-once на рівні доставки не існує ні в кого: at-least-once + ідемпотентний ефект. Ідемпотентність — властивість операції; ключ — інструмент